

Handling Null Values in PySpark DataFrames

A practical, code-first cheat sheet covering detection, dropping, filling, conditional replacement, and full cleaning pipelines for null values — built for medallion-architecture (bronze → silver → gold) workflows.

DataFrame API

fillna / dropna

when / coalesce

- 01 Detecting Nulls
- 02 Dropping Nulls
- 03 Filling Nulls
- 04 Conditional Replacement
- 05 Combining Conditions
- 06 Statistical & Derived Fills
- 07 Full Cleaning Pipeline & Verification

01 Detecting Nulls

Count null values in every column

```
df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

🔗 Loops over every column and counts how many rows are null in each — the standard first move in any EDA pass.

Display rows containing null values

```
from pyspark.sql.functions import col

df.filter(
    col("salary").isNull()
).show()
```

Display rows where department is NOT null

```
df.filter(
    col("department").isNotNull()
).show()
```

02 Dropping Nulls

Drop rows containing any null value

```
df.dropna().show()
```

🔗 Aggressive — drops a row if even one column is null. Use sparingly on wide tables.

Drop rows where salary is null

```
df.dropna(subset=["salary"]).show()
```

🔗 Scoped drop — only checks the listed column(s), leaving nulls elsewhere untouched.

Drop rows having fewer than 6 non-null values

```
df.dropna(thresh=6).show()
```

🔗 thresh keeps a row only if it has at least that many non-null values — useful for sparse records.

03 Filling Nulls

Fill all numeric null values with 0

```
df.fillna(0).show()
```

Fill string null values

```
df.fillna({
  "city": "Unknown",
  "department": "Unknown"
}).show()
```

Fill salary with 5000

```
df.fillna({
  "salary": 5000
}).show()
```

Fill multiple columns at once

```
df.fillna({
  "salary": 5000,
  "bonus": 0,
  "city": "Unknown",
  "department": "Unknown"
}).show()
```

💡 Passing a dict lets you set a different fill value per column in a single call.

04 Conditional Replacement

Replace null salary using when()

```
df2 = df.withColumn(
  "salary",
  when(col("salary").isNull(), 5000)
  .otherwise(col("salary"))
)

df2.show()
```

Replace null department

```
df2 = df.withColumn(
  "department",
  when(col("department").isNull(), "Unknown")
  .otherwise(col("department"))
)
```

Use coalesce() to choose the first non-null value

```
df.select(
  "email",
  coalesce("email", lit("no_email@example.com")).alias("email_new")
).show()
```

💡 coalesce() scans left to right and returns the first non-null expression — a compact alternative to when/otherwise.

05 Combining Conditions

Find employees with both salary and bonus missing

```
df.filter(
  col("salary").isNull() &
  col("bonus").isNull()
).show()
```

Find employees where city OR department is missing

```
df.filter(
  col("city").isNull() |
  col("department").isNull()
).show()
```

Count employees with missing emails

```
df.filter(
  col("email").isNull()
).count()
```

06 Statistical & Derived Fills

Replace null salary with the average salary

```
avg_salary = df.select(avg("salary")).first()[0]

df2 = df.fillna({
  "salary": avg_salary
})
```

💡 Compute the aggregate first, then feed it into fillna() — a common pattern for imputing with the column mean.

Replace null bonus with 0 and create total_income

```
df2 = (
  df.fillna({"bonus": 0})
  .withColumn(
    "total_income",
    col("salary") + col("bonus")
  )
)
```

💡 Chaining fillna() before the derived column avoids nulls silently propagating into the new total_income column.

Verify no nulls remain

```
df2.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df2.columns
]).show()
```

💡 Re-run the same null-count check from Section 01 after cleaning to confirm the fix actually worked.

Fill null values across the full dataset

```
df_clean = df.fillna({
    "salary": avg_salary,
    "bonus": 0,
    "department": "Unknown",
    "city": "Unknown",
    "email": "no_email@example.com"
})
```

Drop rows where full_name is null

```
df_clean = df_clean.dropna(subset=["full_name"])

# Check remaining null values
df_clean.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df_clean.columns
]).show()
```

💡 full_name is treated as non-negotiable — a row without it gets dropped rather than filled, since there's no sane default for a name.